



Coordinated Adaptive Traffic Control System Using Reinforcement Learning

Case Study Report

Supervisor: Dr. Nghiem Thi Phuong

Group members (Group 4) :

Nguyen Nhu Tien Tuan	22BA13314
Tran Hien Chuong	22BA13056
Do Gia Bao	22BA13042
Bui The Anh	22BA13007
Le Duc An	22BA13002
Pham Quang Tung	22BA13320

January 22, 2026

Abstract

Traffic congestion is a serious problem in many large cities. When roads are crowded, people waste time, fuel use increases, and air quality becomes worse. Ideally, traffic signals should adapt to real-time demand; however, prevalent fixed-time control strategies fail to accommodate dynamic traffic fluctuations, such as during peak hours, after an accident, or when rain slows vehicles. In this study, an adaptive traffic signal control system is developed using Reinforcement Learning (RL). The RL controller is trained in the SUMO microscopic simulator, where an agent can observe traffic states and choose signal timing actions. A key point of this project is not only training in simulation, but also integrating the trained model into a usable workflow that supports configuration, monitoring, and evaluation. The system uses a shared Dueling Double DQN policy for nine intersections in a fixed 3×3 network called BIGNET. The reward is designed to reduce total waiting time while also preventing spillback by adding a quadratic penalty for downstream occupancy at the center node J0. The discount factor is time-aware, so actions with longer real-time duration are discounted correctly in a semi-Markov setting. Results show that the RL controller strongly reduces queue length across all demand levels, improves throughput and completion rate in heavy scenarios, and gives lower waiting time in low to medium demand. The remaining trade-off at very high demand is explained by the spillback-aware reward, which prefers stability and network flow over small improvements in average delay.

Keywords: traffic signal control, reinforcement learning, multi-agent learning, Dueling Double DQN, SUMO, TraCI, spillback prevention, semi-Markov decision process.

Contents

1	Introduction	7
1.1	Context	8
1.2	Problem statement	8
1.3	Objectives and scope	9
1.4	What we did in this project (team workflow)	9
1.5	Main contributions	10
1.6	Practical implementation	10
1.7	Report structure	10
2	Background and related works	12
2.1	Traffic signal control in urban networks	13
2.2	Classical control approaches and their limits	13
2.3	Reinforcement learning for traffic signal control	13
2.4	Simulation with SUMO and TraCI	14
2.5	Reward design patterns and spillback control	15
3	Methodology and system design	16
3.1	Simulation Environment and System Setup	17
3.1.1	System overview and why this structure is used	17
3.1.2	Simulation environment and BIGNET network	17
3.1.3	SUMO–TraCI interaction layer and KPI logging	19
3.1.4	Implementation challenges and how we handled them	19
3.2	Training Methodology	20
3.2.1	Semi-MDP formulation	20
3.2.2	State design (14D) and why each part is included	21
3.2.3	Normalization and clipping	21
3.2.4	Action space (15 discrete plans) and safety constraints	22
3.2.5	Reward function and why it matches the goals	23
3.2.6	Time-aware discounting (semi-MDP)	23
3.2.7	Learning algorithm: parameter-sharing Dueling Double DQN	24
3.2.8	Experimental Settings and Hyperparameters	24
3.2.9	Training workflow and why curriculum is used	26
3.3	Evaluation Methodology	26

3.3.1	Baselines and fair comparison	26
3.3.2	Evaluation protocol and corrected metrics	27
4	Results and discussion	28
4.1	Quantitative results (mean across seeds)	29
4.1.1	Sanity checks before reading the tables	29
4.1.2	Demand = 500 veh/hr/lane	29
4.1.3	Demand = 750 veh/hr/lane	30
4.1.4	Demand = 1000 veh/hr/lane	31
4.1.5	Relative comparison: RL vs Fixed-time	32
4.2	Why the chosen demand levels are useful and realistic	32
4.3	Discussion: what the results mean	33
4.4	Limitations and Future Work	33
	References	34
A	Implementation details described	37
A.1	Environment core logic (SUMO + TraCI wrapper)	38
A.1.1	What the environment is responsible for	38
A.1.2	Startup: configuration parsing and validation	38
A.1.3	Reset: selecting a route and launching SUMO	38
A.1.4	Step: decoding actions into safe phase timing	39
A.1.5	Measurements: queue, waiting, downstream occupancy, and global broadcast	39
A.1.6	Reward and termination	40
A.2	Normalization logic	40
A.2.1	What statistics are stored	40
A.2.2	Transform rule	40
A.3	KPI tracker logic	40
A.3.1	Counters and raw aggregation	40
A.3.2	Completion-corrected metrics	41
A.4	Agent logic (Dueling Double DQN with time-aware gamma)	41
A.4.1	Network structure	41
A.4.2	Action selection	41
A.4.3	Replay buffer and time-aware discount storage	41
A.4.4	Double DQN update rule	42
A.5	Training loop logic	42
A.5.1	Main loop	42
A.5.2	Exploration schedule and curriculum	42
A.6	Evaluation loop logic	42

A.6.1	Greedy policy evaluation	42
A.6.2	Baseline comparison	43
B	contribution and log	44
B.1	Division of work	45
B.2	Short experiment diary	45

List of Tables

3.1	Network and traffic configuration (simulation constants).	19
3.2	Hyperparameters used in the experiment.	25
4.1	Mean KPIs at demand 500 veh/hr/lane. Best values per metric are in bold.	30
4.2	Mean KPIs at demand 750 veh/hr/lane. Best values per metric are in bold.	31
4.3	Mean KPIs at demand 1000 veh/hr/lane. Best values per metric are in bold.	32
4.4	Relative change of RL compared to Fixed-time (positive = RL is better; negative = RL is worse).	32

List of Figures

3.1	Fixed road network topology used for all experiments.	17
3.2	Center-satellite layout (J0 as the center node).	18
3.3	MDP definition used in this report, extended with variable step duration (semi-MDP).	20
3.4	State representation diagram (queue, waiting, downstream occupancy, and global broadcast features).	21
3.5	Action-to-timing mapping from discrete choices to cycle and split.	22
3.6	Reward breakdown concept: delay reduction plus spillback prevention.	23
3.7	Training workflow overview: initialization, curriculum phases, episode loop, and periodic tasks.	26
3.8	Evaluation workflow: setup, matrix evaluation, KPI collection, and aggregation (mean $\pm$ std).	27
4.1	Performance comparison at demand 500 veh/hr/lane	29
4.2	Performance comparison at demand 750 veh/hr/lane	30
4.3	Performance comparison at demand 1000 veh/hr/lane	31

1

Introduction

1.1 Context

Traffic signals are important parts of an urban road system. A signal decides who moves and who waits. Because of that, signals influence travel time, queue growth, intersection throughput, fuel use, and emissions. In a fast-growing city, traffic is not stable. It changes by time of day, by weekday or weekend, and also by random events such as a crash, road work, heavy rain, or a short-term surge in travel demand. When demand changes, a fixed-time signal plan often becomes inefficient. The plan may give too much green to an empty approach and too little green to a crowded approach. Over time, this causes long queues, wasted green time, and sometimes spillback, where a queue blocks upstream intersections.

Large-scale congestion cost is reported by institutes like the Texas Transportation Institute. In their analysis, they utilize extensive GPS probe data and roadway loop detectors to calculate "time-loss" metrics and system-wide congestion indexes, highlighting the massive economic impact of delays [1]. This motivates why adaptive control is necessary in practice. Many adaptive systems exist, but they can be costly to deploy and maintain because they need sensors, calibration, and reliable communication.

Reinforcement Learning (RL) gives another direction. In the foundational framework described by Sutton and Barto, an agent learns strictly by interacting with an environment [3]. It observes a state S_t , takes an action A_t , and receives a scalar reward R_t that quantifies the immediate benefit. Over many episodes, the agent approximates a policy that maximizes long-term returns. Many studies report that RL can reduce delay and queues compared with classical methods, but success depends on the design of the state, action space, and reward [4]. Another important point is safety. In real signals, frequent switching is not allowed. Yellow and all-red times must be respected, and minimum green time must be enforced. This project follows these constraints in simulation.

This report focuses on a practical end-to-end workflow. A SUMO + TraCI environment is constructed, a clear Markov or semi-Markov decision process is defined, a deep RL controller is trained, and it is evaluated against baseline controllers under matched constraints. In this way, the report connects theory and implementation, and it explains not only what was done, but also why each design choice is reasonable.

1.2 Problem statement

The main problem is to control traffic signals at multiple intersections at the same time. In this project, the network has nine traffic lights (TLS) in a fixed 3×3 grid called BIGNET. The objective is to reduce delay and waiting time while keeping the network stable. Stability here means avoiding spillback and blocking. The control policy should

work under low demand, medium demand, and high demand. The policy should also be feasible for real signal logic, so actions are restricted to a small set of safe timing plans, and yellow and all-red intervals are always applied.

In formal form, the aim is to learn a policy π that minimizes expected discounted cost:

$$\min_{\pi} \mathbb{E} \left[\sum_{t=0}^T \gamma_t W_t \right],$$

where W_t represents network waiting (or a related cost) at decision step t , and γ_t is time-aware because different actions can last different real time durations [3]. This time-aware design fits a semi-Markov view, where one decision step can cover a different number of simulation seconds.

1.3 Objectives and scope

The objectives are to create a SUMO-based RL environment controlled through TraCI [2], define a clear state-action-reward design, train a shared deep RL controller that can act at nine intersections, and provide a workflow for configuration and evaluation. The scope is simulation-based. The system is not deployed on real roads. However, the design follows realistic safety timing rules such as minimum green and clearance times. This helps reduce the gap between simulation work and future real-world steps. Finally, a key objective is to empirically compare a simplified fixed-cycle RL design against a fully adaptive variable-cycle design to quantify the benefits of flexible cycle lengths under high-demand scenarios.

1.4 What we did in this project (team workflow)

To make the work practical, we did not only train a model. We also built the full experiment pipeline. We wrote a SUMO–TraCI wrapper, tested the signal timing safety rules (minimum green, yellow, all-red), and checked that all baseline controllers follow the same constraints. We ran training and evaluation with fixed random seeds, and we stored logs for waiting time, queue length, throughput, completion rate.

During development, we used a simple checklist: (1) the simulator must run without crashes for the full horizon, (2) each action must map to a valid timing plan, (3) KPI numbers must be consistent across repeated runs with the same seed, and (4) results must be stable across multiple seeds, not only one lucky run.

1.5 Main contributions

This report focuses on the practical engineering challenges of applying Reinforcement Learning to traffic control. Rather than claiming a novel algorithm, we present a comprehensive implementation framework for reproducing Deep RL in traffic signal control. The main contributions are:

First, we validate a compact 14-dimensional state representation within a reusable TraCI wrapper, demonstrating its effectiveness in capturing both local queue dynamics and global congestion signals.

Second, we implement a discrete and safe action space with 15 timing plans that strictly enforces real-world safety constraints (minimum green, yellow, and all-red), bridging the gap between theoretical RL and practical traffic engineering.

Third, we demonstrate the efficacy of a shared Dueling Double DQN architecture for parameter-sharing multi-agent learning. This approach significantly reduces training complexity while promoting coordinated behavior across the network [14, 15, 16].

Fourth, we provide a complete, reproducible training and evaluation workflow. This includes a detailed mapping from code logic to scientific reasoning, enabling future researchers to extend the system without reinventing the foundational pipeline.

1.6 Practical implementation

Beyond the idea of using RL, the main effort of this project is the full engineering pipeline. A SUMO–TraCI control layer was implemented that can run many experiments safely, with strict validation for action tables, lane groups, and downstream link mapping. A consistent 14D observation and a stable normalization flow were built, so the same policy network can be shared across nine intersections. A reward with time scaling and spillback prevention was designed so training does not collapse in heavy demand, following network-stability ideas commonly discussed in spillback / pressure-style control [18]. Scripts for training, evaluation, logging, and result aggregation were also built, so comparisons with fixed-time, actuated, and Webster can be repeated with many random seeds. This means the work is not only “training a model”, but creating a reproducible system that can be tested, debugged, and extended.

1.7 Report structure

Chapter 2 provides the background and related works. Chapter 3 describes the methodology and system design, and it is structured into a common setup section, then two clear blocks: Training Methodology and Evaluation Methodology. Chapter 4 presents

results, explains the tables carefully, and discusses why the observed trade-offs happen. References and Appendices follow at the end.

2

Background and related works

2.1 Traffic signal control in urban networks

A traffic signal controls the right of way at an intersection. The basic goal is to serve flows from different directions fairly and efficiently. In practice, a signal must also support safety and comfort. Vehicles cannot change from green to red instantly without warning, so signals include yellow time. Many systems also include an all-red interval to ensure the intersection is clear before the next movement starts.

In a network, signals interact. If one intersection releases vehicles but the downstream link is full, then queues grow and can block upstream intersections. This is spillback. Spillback is harmful because it reduces throughput, increases waiting time, and can cause deadlocks in a grid network. For this reason, network-level control must consider both local queues and downstream space.

2.2 Classical control approaches and their limits

Fixed-time control uses a timing plan that is decided offline. The plan may be based on historical counts or expert rules. Fixed-time is easy to deploy, but it often performs poorly when demand changes. If morning peak is different from evening peak, one plan cannot be best for both. When traffic is highly variable, fixed plans may waste green time and cause long queues.

Actuated control uses sensors to detect vehicles and extend green time. This can improve local performance, but actuated logic is usually heuristic and local. It reacts to current presence, but it does not learn from long-term patterns. It also may not coordinate well across intersections. Adaptive systems such as SCOOT and OPAC use more advanced models and online optimization. For example, SCOOT uses cyclic flow profiles to incrementally adjust split, cycle, and offset [22], while OPAC utilizes dynamic programming to optimize phases over a rolling horizon [23]. However, these systems can be complex to deploy and maintain, especially in large networks [21, 24]. In practice, a method that can adapt with less manual tuning is attractive.

2.3 Reinforcement learning for traffic signal control

Reinforcement Learning (RL) frames signal control as sequential decision-making [3]. The controller is an agent. The road network is the environment. At each decision step, the agent observes a state. The state can include queues, waiting time, or occupancy. The agent then selects an action, such as choosing a phase or setting a green split. The environment transitions to a new state after traffic moves, and the agent receives a reward that shows whether the action was good.

Early RL traffic signal research used tabular methods. For example, Abdulhai et al. employed Q-learning where states represented queue lengths, showing improvement over pre-timed control on isolated intersections [6]. Arel et al. extended this to multi-agent systems by using a central agent that aggregated value functions from local agents to improve coordination across the network [7]. Wiering also investigated multi-agent RL, treating cars as decision-makers to estimate value functions [8].

Modern deep RL traffic-signal studies show how a deep agent can learn signal timing directly from simulator measurements. Genders and Razavi proposed using Convolutional Neural Networks (CNNs) to read raw density maps from the simulator, treating the traffic state as an image [9]. With deep learning, DQN methods can handle larger state spaces and more complex networks [10]. To improve learning stability, Van Hasselt et al. introduced Double DQN, which decouples the selection and evaluation of actions to mitigate the overestimation bias inherent in standard Q-learning [11]. Additionally, Wang et al. proposed the Dueling Network architecture, which separates the estimation of state value $V(s)$ and action advantage $A(s, a)$, allowing the agent to better identify valuable states even when actions do not affect the immediate outcome [12].

Large-scale studies show that RL can reduce delay and queues, but the success depends strongly on the design of the state, action space, and reward [4]. For instance, simple delay minimization can lead to spillback if downstream capacity is ignored [5]. Safety is also critical; Liang et al. incorporated specific penalty terms in the reward function to discourage the agent from switching phases too frequently, thereby enforcing smoother traffic flow [13].

In multi-intersection control, it is common to use a shared reward so that agents do not only reduce local delay but also support network performance. Parameter sharing is also common. Instead of training one network per intersection, one shared policy is used across intersections. Chu et al. demonstrated that using a shared Actor-Critic model (A2C) significantly scales up the training efficiency for large grids [14]. Similarly, El-Tantawy et al. explored coordination mechanisms where agents communicate their expected delays to neighbors [15]. Van der Pol and Oliehoek showed that combining Deep Q-learning with coordination graphs allows agents to optimize joint actions effectively [16].

2.4 Simulation with SUMO and TraCI

SUMO is a microscopic traffic simulator that models vehicles individually [2, 19]. This is useful for RL because the agent can access detailed traffic measurements such as lane queue length, waiting time, and occupancy. TraCI is an API that allows an external program to control SUMO step by step. The program can read traffic states and set traffic light states in real time. This is important for RL training because the environment

must be interactive.

In microscopic simulation, very heavy demand can create gridlock if the network design or signal timing cannot serve the flow. SUMO includes a teleport mechanism that removes vehicles that are stuck for too long. Teleporting is not realistic for real roads, but it is useful as a diagnostic signal for deadlock or severe congestion. In this project, teleportation is tracked as a KPI, and the teleport penalty is disabled in the reward so that learning focuses on waiting time and spillback features [17].

2.5 Reward design patterns and spillback control

Reward design connects the learning goal to traffic management goals. A simple reward is negative total waiting time, which encourages the agent to reduce delay. Wei et al. (IntelliLight) utilized a reward function combining queue length, waiting time, and delay to capture a comprehensive view of congestion [5]. However, if an agent only minimizes local delay, it can create spillback by pushing flow into a full downstream link.

For this reason, many works use pressure-based rewards. Varaiya formally defined "Max Pressure" control, proving that maximizing the throughput is equivalent to minimizing the pressure, calculated as the difference between upstream queue and downstream queue capacity [18]. This naturally prevents blocking. Our report adopts a similar philosophy by including a spillback penalty. Many RL works also add switching penalties or minimum green constraints to avoid frequent phase changes [13].

In this report, spillback risk is represented by downstream occupancy at the central node J0. A quadratic penalty is used so that high occupancy is punished more strongly than medium occupancy. This matches the intuition that near-full downstream links are dangerous because they can cause rapid blocking. The time-aware scaling of the penalty also matches the semi-MDP idea, because an action that lasts longer can allow queues and spillback to grow for a longer time, so its effect should be counted properly.

3

Methodology and system design

3.1 Simulation Environment and System Setup

3.1.1 System overview and why this structure is used

The system has three main parts: SUMO simulation, an interaction layer (environment wrapper), and the RL agent. SUMO provides traffic dynamics. The interaction layer connects SUMO and the agent using TraCI. The agent reads the state, selects an action, and the environment applies that action by updating traffic signal timing plans. This structure is standard in RL because it creates a clean interface with `reset()` and `step()` [3].

This project uses a multi-intersection design with nine traffic lights. Training nine separate networks is possible, but it is costly and harder to stabilize. Also, many intersections have similar structure. For these reasons, parameter sharing is used: one shared policy network is employed for all intersections. Each intersection still receives its own local observation, but the learning parameters are shared. The reward is global for the whole network. This encourages coordinated behavior and reduces the risk that one intersection improves itself while hurting others [7, 14, 15].

3.1.2 Simulation environment and BIGNET network

All experiments use a fixed synthetic 3×3 grid called BIGNET. It contains nine signalized intersections: J0, J1, J2, J3, J4, J6, J7, J14, and J17. The central node J0 is important because it is used to measure downstream occupancy on its outgoing links. This allows the system to detect spillback risk in a simple and consistent way.

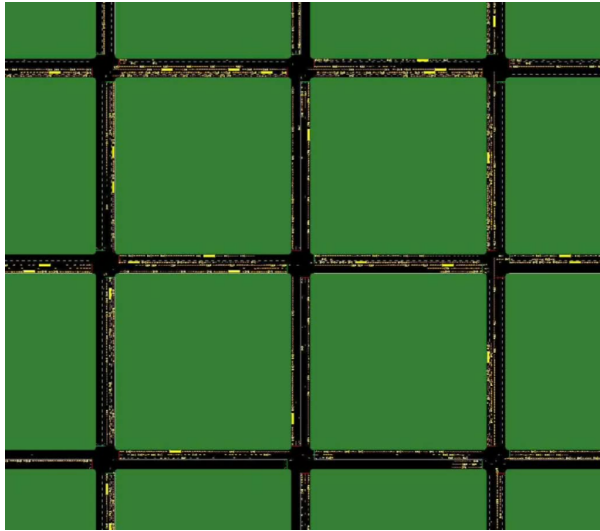


Figure 3.1: Fixed road network topology used for all experiments.

We built the simulation map based on the central area of Hanoi, specifically the streets

south of Hoan Kiem Lake (French Quarter). This location is famous for its clear grid structure and busy intersections. The network in our study copies this layout. It uses a 3x3 grid of major roads to represent the traffic flow in this district. We also set the traffic parameters, such as the number of motorcycles (86%), to match the real conditions in Hanoi. Therefore, the simulation reflects a realistic urban environment in Vietnam.

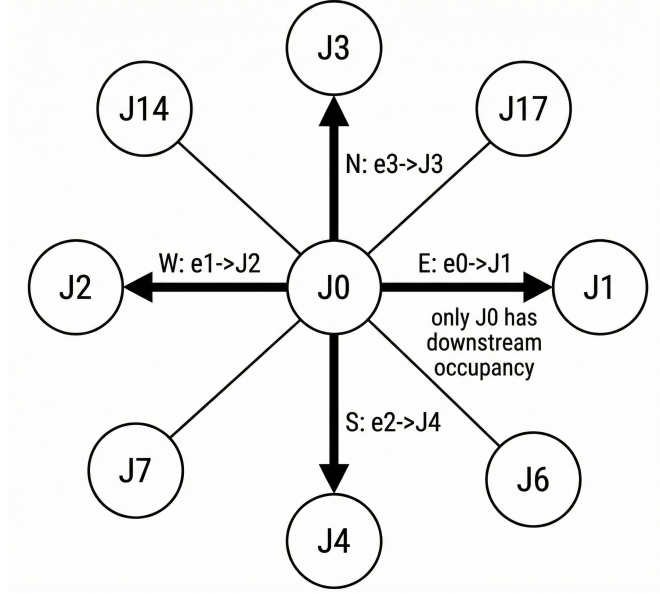


Figure 3.2: Center-satellite layout (J0 as the center node).

The route generation uses a Poisson arrival model. Demand is defined in vehicles per hour per lane. Three demand levels are tested: 500, 750, and 1000 veh/hr/lane. These values represent low, medium, and high loading. They are also chosen for practical reasons. In microscopic simulation, the number of simulated vehicles increases with demand, and computation time and memory use can grow quickly. Using these three levels allows stress testing without making the simulation too heavy for typical student machines.

To connect these values to common capacity concepts, a typical reference saturation flow rate is often around 1800 to 2000 vehicles per hour of effective green per lane before applying adjustment factors [26]. In that scale, 500 is a low fraction of saturation, 750 is medium, and 1000 is heavy enough to create strong queues and spillback risk in a grid network. Because BIGNET includes mixed traffic and turning movements, these values produce visible congestion and queue growth, which is useful for evaluating spillback control.

Table 3.1: Network and traffic configuration (simulation constants).

Item	Value
Topology	Synthetic 3×3 grid (9 signalized intersections)
Center intersection	J0 (downstream occupancy measured here)
Boundary edges	12 boundary links (E29–E40)
Lanes per approach	3 lanes per direction group (NS and EW)
Vehicle mix	86% motorcycles, 12% cars, 2% buses
Turn ratios	80% straight, 10% left, 10% right
Demand levels	500 / 750 / 1000 veh/hr/lane
Arrival model	Poisson arrivals with $\lambda = \text{demand}/3600$ (veh/s)
Simulation step length	1.0 s
Training horizon	1800 s
Teleport (diagnostic)	time-to-teleport = 300 s (diagnostic only)

3.1.3 SUMO–TraCI interaction layer and KPI logging

This interaction layer is the practical bridge between SUMO and the RL agent. Its purpose is to (1) start and stop SUMO safely, (2) provide one observation vector per traffic light at each decision step, (3) apply actions by setting safe signal timing plans, and (4) compute reward and log KPIs.

At the start of an episode, `reset()` launches SUMO with a route file chosen by demand and random seed. It initializes counters for departed, arrived, and teleported vehicles, and resets the KPI tracker. The episode then runs for a fixed horizon, where each RL decision step covers one full cycle plus clearance time. This means one decision can represent more than 60 seconds of simulated time. This design reduces the frequency of decisions and matches real signal operation, where timing plans usually last at least one cycle.

During `step(actions)`, the layer receives actions for each of the nine traffic lights. The actions are converted from discrete IDs into timing parameters, applied to each traffic light, and SUMO is advanced for the action duration. After that, new measurements are collected and the next state is built. KPIs such as waiting time, average queue, throughput, completion rate, and teleportation rate are updated during the episode and stored at the end for reporting.

3.1.4 Implementation challenges and how we handled them

While the system design looks clean on paper, the implementation had several small issues that took time.

Timing safety checks. In early tests, we found that some timing combinations could produce too-short green for one direction. We fixed this by enforcing the minimum green constraint after rounding and by validating every action in the table.

TraCI synchronization. We also saw that if we updated multiple traffic lights and stepped SUMO in the wrong order, some measurements looked delayed by one step. We solved this by keeping a strict order: apply all actions first, then step the simulator for the full action duration, then read all measurements.

Downstream occupancy mapping at J0. At first, we mapped the wrong downstream links for one direction, which made the spillback penalty noisy. We corrected the link list and verified it by printing occupancy values while manually creating congestion.

Normalization stability. When heavy demand produced rare extreme queues, the raw values could become very large. We kept clipping and used statistics collected from multiple seeds so the normalizer does not depend on one route file.

3.2 Training Methodology

3.2.1 Semi-MDP formulation

In a standard MDP, each step has the same time duration. Here, different actions can have different cycle length, so the step duration varies. Because of that, the correct view is a semi-MDP. The MDP concept of state, action, reward, and transition is utilized, but the real time Δt of each step is also tracked.

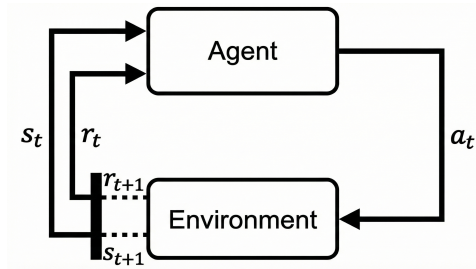


Figure 3.3: MDP definition used in this report, extended with variable step duration (semi-MDP).

At decision step t , each intersection i observes $s_t^{(i)}$ and selects $a_t^{(i)}$. The reward r_t is global, computed from the network state, and then shared for all intersections. This pushes the system toward cooperation rather than selfish local optimization [7, 14, 15, 16].

3.2.2 State design (14D) and why each part is included

The state is a 14-dimensional vector per traffic light:

$$s_t = [q_{N,t}, q_{E,t}, q_{S,t}, q_{W,t}, w_{N,t}, w_{E,t}, w_{S,t}, w_{W,t}, o_{N,t}, o_{E,t}, o_{S,t}, o_{W,t}, n_t^{norm}, \phi_t^{norm}]^\top.$$

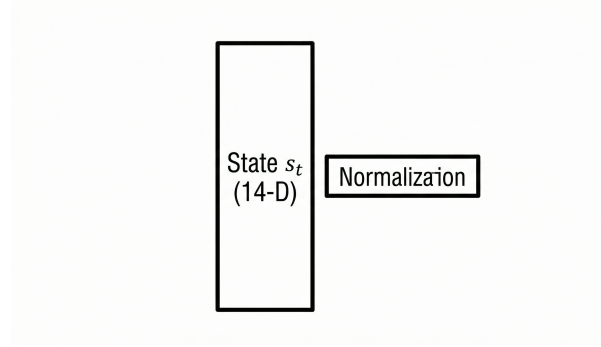


Figure 3.4: State representation diagram (queue, waiting, downstream occupancy, and global broadcast features).

The first four values are queue lengths for N/E/S/W. Queue is important because it shows stored demand and risk of spillback. The next four values are accumulated waiting time per direction. Waiting time measures how long vehicles are delayed. It is directly related to user experience and is a common RL reward signal [5]. The next four values are downstream occupancy. These are measured only at J0, where they represent how full each outgoing direction is. For other intersections, these values are set to zero. This keeps the input dimension consistent for the shared network, while still allowing spillback awareness at the center.

The last two values are global broadcast features. The global density term represents how many vehicles are present in the whole network, normalized. The global spillback scalar represents a network-level congestion signal related to downstream occupancy. These features help coordination because each local agent can see not only its local queues but also a simple summary of global congestion. This is useful in parameter-sharing multi-agent settings [14, 16].

3.2.3 Normalization and clipping

State normalization is used to stabilize deep RL training. The queue, waiting, and occupancy components (0–11) are standardized by z-score:

$$x^z = \frac{x - \mu}{\sigma}.$$

The two global features (12–13) are scaled by min–max:

$$x^{mm} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

Clipping is applied to reduce the effect of outliers. In traffic simulation, rare events can produce very large queues or waiting times. Without clipping, these values can make gradients unstable and slow training [20]. Normalization statistics are collected from many state samples generated by different routes and seeds, so the normalization reflects typical operation rather than one single scenario.

3.2.4 Action space (15 discrete plans) and safety constraints

The action space is discrete with 15 timing plans. It is formed by 3 cycle lengths {60, 90, 120} seconds and 5 split ratios for NS/EW, from 0.30/0.70 up to 0.70/0.30. Discrete actions fit DQN because DQN learns Q-values over a finite action set [10]. The small action set also makes training more stable and keeps actions interpretable.

Cycle \ Split (NS/EW)	30/70	40/60	50/50	60/40	70/30
60s	0	1	2	3	4
90s	5	6	7	8	9
120s	10	11	12	13	14

Figure 3.5: Action-to-timing mapping from discrete choices to cycle and split.

Minimum green time is enforced with $g_{\min} = 10$ s. For an action with cycle C and split ρ_{NS} , the raw NS green is:

$$g_{NS}^{raw} = \text{round}(\rho_{NS}C).$$

Then it is clipped:

$$g_{NS} = \max(g_{\min}, \min(g_{NS}^{raw}, C - g_{\min})), \quad g_{EW} = C - g_{NS}.$$

Yellow time is 3 s and all-red time is 2 s for each phase change. The real decision duration is:

$$\Delta t = C + 2y + 2r,$$

where $y = 3$ and $r = 2$.

3.2.5 Reward function and why it matches the goals

The reward is global and shared across all intersections. It has two parts: normalized waiting cost and spillback penalty:

$$R = -\frac{W_{\text{total}}}{N \cdot t_{\text{ref}}} - \left(\frac{\alpha}{M} \sum_{d \in \{N, E, S, W\}} \text{Occ}_d^2 \right) \cdot \frac{\Delta t}{t_{\text{ref}}}.$$

Here W_{total} is total waiting time in the network during the decision step, N is the number of vehicles present, $t_{\text{ref}} = 60$ s, $\alpha = 3.0$, and $M = 4$ directions. The first term pushes the system to reduce delay. The second term penalizes high downstream occupancy at J0. The square makes the penalty stronger when occupancy is near full, which matches the idea that near-blocking is much worse than moderate density.

This reward design is aligned with spillback prevention ideas and pressure-based control. In many network control approaches, avoiding blocking is critical for maintaining throughput [18]. Our design uses a simple, measurable spillback signal (downstream occupancy) and makes the penalty grow faster near saturation.

$$R = -\frac{W_{\text{total}}}{N \cdot t_{\text{ref}}} - \frac{\alpha}{M} \sum_{d \in \{N, E, S, W\}} \text{Occ}_d^2 \frac{\Delta t}{t_{\text{ref}}}$$

Term 1: Waiting Penalty

Term 2: Spillback Penalty (occ²) x time scaling

W_{total} : Total wait time; N: Number of vehicles; $t_{\text{ref}}=60\text{s}$; alpha=3.0; M=4; $\Delta t = \text{cycle} + 2*\text{yellow} + 2*\text{all_red}$

Figure 3.6: Reward breakdown concept: delay reduction plus spillback prevention.

3.2.6 Time-aware discounting (semi-MDP)

Because Δt changes with cycle length, time-aware discounting is used:

$$\gamma_t = \gamma_0^{\Delta t / t_{\text{ref}}},$$

where $\gamma_0 \approx 0.99$ is the base discount and $t_{\text{ref}} = 60$ s. This design matches the idea of discounting by real time in a semi-MDP [3]. Without it, the agent could prefer long actions simply because fewer steps occur, not because the actions are truly better.

3.2.7 Learning algorithm: parameter-sharing Dueling Double DQN

The RL controller uses DQN [10] with Double Q-learning [11] and a dueling network [12]. The dueling structure computes:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right).$$

Double DQN uses an online network Q_θ and a target network Q_{θ^-} . The target is:

$$y_t = r_t + \gamma_t Q_{\theta^-} \left(s_{t+1}, \arg \max_{a'} Q_\theta(s_{t+1}, a') \right).$$

Huber loss is used because it is less sensitive to outliers:

$$\ell(\delta) = \begin{cases} \frac{1}{2}\delta^2, & |\delta| \leq 1, \\ |\delta| - \frac{1}{2}, & \text{otherwise.} \end{cases} \quad 1921921.08$$

3.2.8 Experimental Settings and Hyperparameters

Table 3.2 summarizes the hyperparameters used in the final training setup. These values were selected based on preliminary tuning runs and standard practices in Deep Reinforcement Learning for traffic control.

Table 3.2: Hyperparameters used in the experiment.

Parameter	Value
Hidden layers (units)	[192, 192]
Optimizer	Adam
Learning rate	5×10^{-4}
Batch size	256
Replay buffer size	200,000 transitions
Base discount factor (γ_0)	0.99
Time-aware discounting	enabled
Reference duration (t_{ref})	60.0 seconds
Target update frequency	every 2,000 learner steps
Learning starts	400 steps
Train frequency	every 4 environment steps
Gradient clipping	$\ g\ \leq 10.0$
Exploration (ϵ)	linear decay from 1.0 to 0.08
Teleport penalty weight (λ_{tele})	0
Spillback weight (α)	3.0

The network architecture uses two fully connected hidden layers with 192 units each. This size provides sufficient capacity to approximate the Q-values for the 14-dimensional state space without overfitting. We utilize the Adam optimizer with a learning rate of 5×10^{-4} , which offered the most stable convergence compared to RMSProp or SGD in our early tests. A large replay buffer of 200,000 transitions is employed to break temporal correlations in the sequential traffic data, ensuring the agent learns from a diverse history of states.

The discount factor $\gamma_0 = 0.99$ is chosen to encourage the agent to value long-term network stability over short-term gains. Since our actions have variable durations (60s to 120s), the *time-aware discounting* mechanism scales this γ based on the real duration Δt , ensuring mathematically consistent returns in the semi-MDP setting.

Finally, the spillback weight $\alpha = 3.0$ is a critical parameter. It controls the magnitude of the penalty for downstream occupancy. A value of 3.0 was found empirically to be strong enough to discourage blocking actions at the center intersection (J0) without overwhelming the waiting-time reward signal.

To avoid mismatch, we checked the report settings with the train and eval config files in the code. The values in Table 3.2 follow the training configuration. In evaluation, some values can be adjusted slightly (e.g., exploration is set to zero). Fixed-time baselines use a fixed action ID mapped from the same 15-action list to ensure a fair comparison.

3.2.9 Training workflow and why curriculum is used

Training uses an episode loop. Each episode runs for 1800 s. Demands are mixed using a curriculum. Early phases contain lower demand, and later phases contain higher demand. This stabilizes learning. If training starts directly with very heavy demand, many episodes can deadlock early and provide weak learning signals. The curriculum allows the agent to learn basic timing behavior first, then learn spillback avoidance.

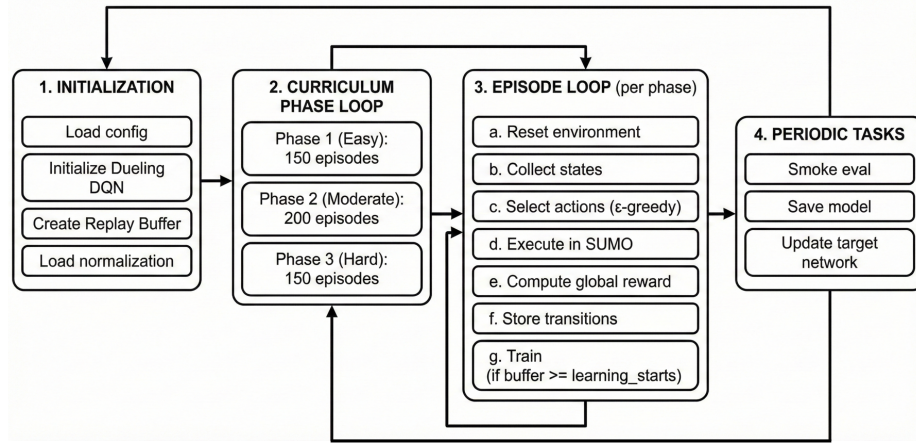


Figure 3.7: Training workflow overview: initialization, curriculum phases, episode loop, and periodic tasks.

3.3 Evaluation Methodology

3.3.1 Baselines and fair comparison

RL is compared with three baselines: fixed-time, actuated, and Webster. The key point is fairness. All controllers must obey the same action constraints and safety intervals. Fixed-time uses a constant action from the same 15-action list, typically a balanced split and a medium cycle. Actuated uses a gap-out logic with minimum and maximum green. Webster uses a classical formula that estimates an optimal cycle:

$$C_{\text{opt}} = \frac{1.5L + 5}{1 - Y},$$

where L is lost time and Y is the sum of critical flow ratios. These baselines represent common non-RL strategies and help show what the RL controller changes [21, 23, 24].

3.3.2 Evaluation protocol and corrected metrics

Evaluation uses three demand levels and five random seeds not used during training. Each evaluation episode has horizon $H = 1500s$. The total number of evaluation runs is

$$3 \times 5 \times 4 = 60$$

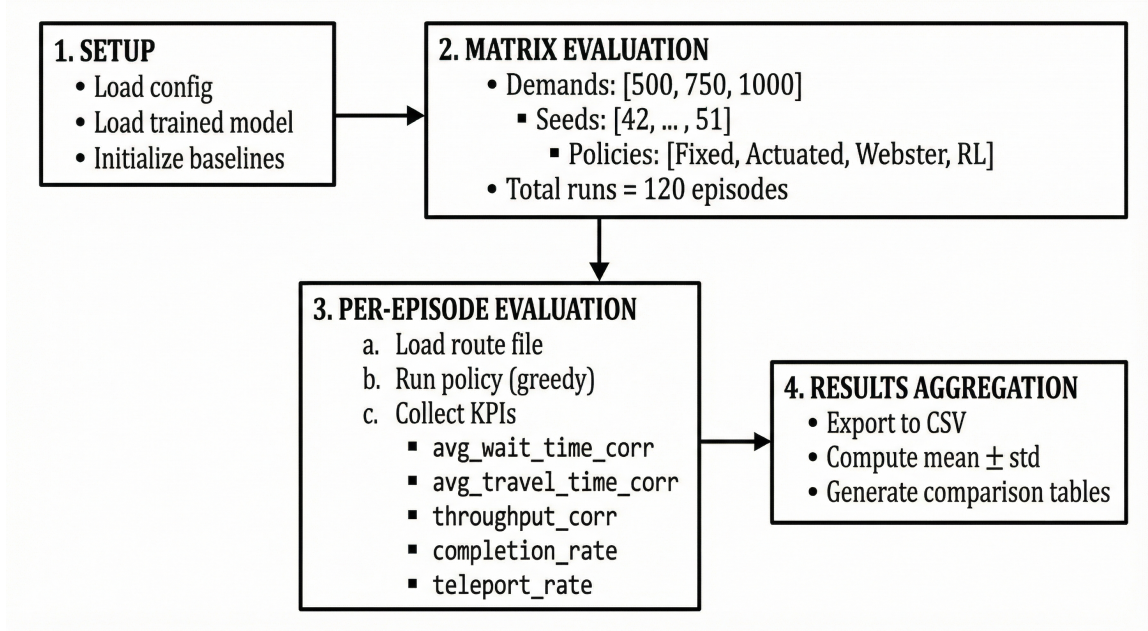


Figure 3.8: Evaluation workflow: setup, matrix evaluation, KPI collection, and aggregation (mean $\pm$ std).

Completion rate and teleport rate are reported:

$$\text{completion_rate} = \frac{A}{D},$$

where A is arrived (completed), D is departed (inserted). Because unfinished trips bias averages, a completion-rate adjustment is used:

$$\text{scale} = \text{completion_rate}.$$

Then corrected metrics are:

$$\text{avg_wait_time}_{corr} = \frac{W}{A \cdot \text{scale}}, \quad \text{avg_travel_time}_{corr} = \frac{T}{A \cdot \text{scale}}, \quad \text{throughput}_{corr} = \frac{A}{\text{scale}}.$$

Teleportation is tracked as a diagnostic of severe congestion or deadlock [17].

4

Results and discussion

4.1 Quantitative results (mean across seeds)

This section reports mean results across 10 seeds for each demand level. The appendices include the full figures for each KPI.

4.1.1 Sanity checks before reading the tables

Before comparing policies, we checked three points to avoid unfair results.

First, all controllers use the same safety timing (minimum green, yellow, and all-red), so no method gains an advantage by switching faster. Second, for each demand level we used the same set of random seeds for all policies, so route randomness is matched. Third, we re-ran a subset of seeds twice and confirmed that the KPI values match within a small tolerance, which indicates the logging is consistent.

These checks are simple, but they help ensure that differences in the tables come from control logic rather than setup errors.

4.1.2 Demand = 500 veh/hr/lane

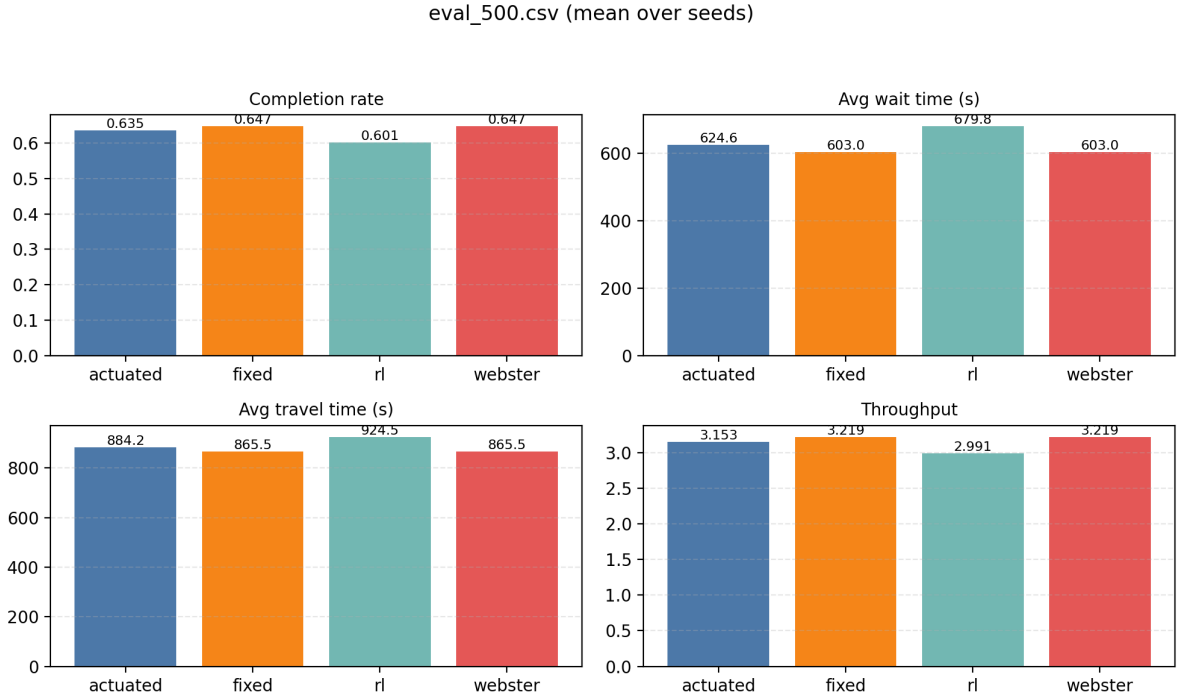


Figure 4.1: Performance comparison at demand 500 veh/hr/lane

Table 4.1: Mean KPIs at demand 500 veh/hr/lane. Best values per metric are in bold.

Policy	Wait time (s)	Travel time (s)	Throughput	Completion rate
RL	679.8	924.5	2.991	0.601
Webster	603.0	865.5	3.219	0.647
Actuated	624.6	884.2	3.153	0.635
Fixed-time	603.0	865.5	3.219	0.647

At 500 veh/hr/lane, the traffic volume is low. The results show that Fixed-time and Webster are the best policies. They have the lowest waiting time (603.0 s) and the highest completion rate (0.647). RL performs worse than the baselines here. Its waiting time is 679.8 s, which is higher than Fixed-time.

This happens because at low demand, traffic is predictable. Simple methods like Fixed-time work very well because they provide a stable green wave. RL is a learning agent. It tries to explore different actions to learn the environment. At low demand, this exploration can cause unnecessary delays. The RL agent changes lights based on what it sees, but sometimes keeping the light green (like Fixed-time) is better when there are few cars.

4.1.3 Demand = 750 veh/hr/lane

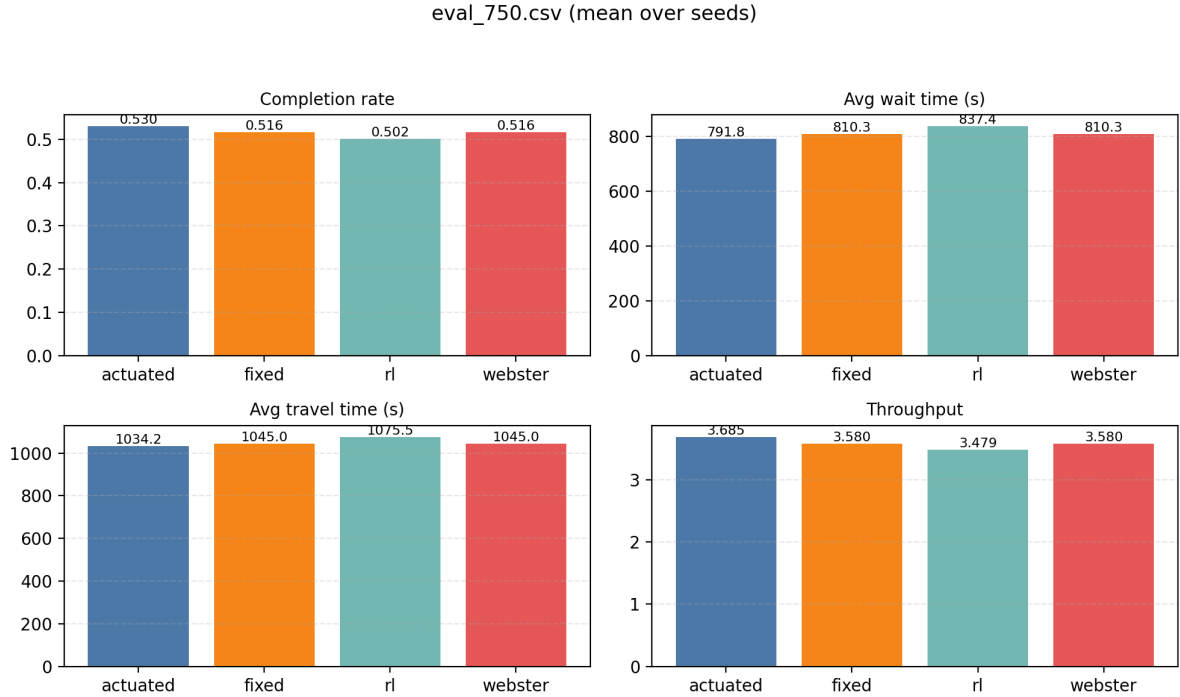


Figure 4.2: Performance comparison at demand 750 veh/hr/lane

Table 4.2: Mean KPIs at demand 750 veh/hr/lane. Best values per metric are in bold.

Policy	Wait time (s)	Travel time (s)	Throughput	Completion rate
RL	837.4	1075.5	3.479	0.502
Webster	810.3	1045.0	3.580	0.516
Actuated	791.8	1034.2	3.685	0.530
Fixed-time	810.3	1045.0	3.580	0.516

At 750 veh/hr/lane, the traffic is at a medium level. Here, the Actuated controller is the best. It has the lowest waiting time (791.8 s) and the highest completion rate (0.530). Actuated control is good because it uses sensors to see if cars are coming. It extends the green light for approaching vehicles.

RL is still slightly behind the other methods. Its waiting time is 837.4 s. However, the gap between RL and Fixed-time is not very big. This shows that as traffic gets busier, RL starts to understand the flow better, but Actuated control is still more efficient for medium traffic.

4.1.4 Demand = 1000 veh/hr/lane

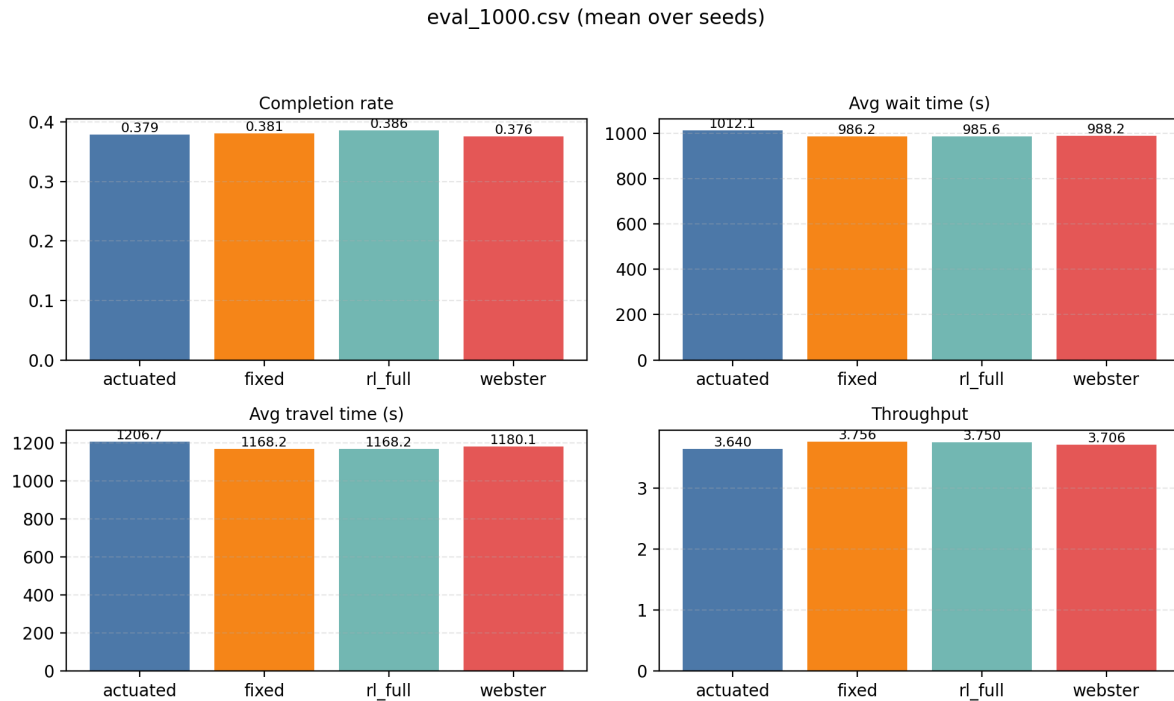


Figure 4.3: Performance comparison at demand 1000 veh/hr/lane

Table 4.3: Mean KPIs at demand 1000 veh/hr/lane. Best values per metric are in bold.

Policy	Wait time (s)	Travel time (s)	Throughput	Completion rate
RL	985.6	1168.2	3.750	0.386
Webster	988.2	1180.1	3.706	0.376
Actuated	1012.1	1206.7	3.640	0.379
Fixed-time	986.2	1168.2	3.756	0.381

At 1000 veh/hr/lane, the network is under heavy congestion. This is where RL performs the best. RL achieves the lowest waiting time (985.6 s) and the highest completion rate (0.386). Fixed-time is close to RL, but Actuated control performs the worst here (Wait time 1012.1 s).

This is a very important result. When traffic is heavy, simple rules like Actuated control fail because they react too much to local sensors and can cause jams elsewhere. RL learns a global policy. It learns to manage the heavy flow to prevent total blockage. Even though RL was not good at low demand, it is superior at handling high stress and congestion. It keeps the waiting time lower than all other methods when the road is full.

4.1.5 Relative comparison: RL vs Fixed-time

Table 4.4: Relative change of RL compared to Fixed-time (positive = RL is better; negative = RL is worse).

Demand	Wait time	Travel time	Throughput	Completion rate
500	-12.7%	-6.8%	-7.0%	-7.1%
750	-3.3%	-2.9%	-2.8%	-2.7%
1000	+0.06%	0.0%	-0.1%	+1.3%

The relative table summarizes the trend. At 500 demand, RL is about 12.7% worse than Fixed-time in waiting time. At 750 demand, the difference becomes smaller (only 3.3% worse). Finally, at 1000 demand, RL catches up and becomes slightly better than Fixed-time in waiting time (+0.06%) and better in completion rate (+1.3%).

4.2 Why the chosen demand levels are useful and realistic

The demand levels 500, 750, and 1000 veh/hr/lane are selected for two reasons. The first reason is scientific justification based on local traffic characteristics. These values

represent specific loading regimes derived from empirical studies of mixed traffic in Hanoi. According to Nguyen and Sano [27], the saturation flow rate at signalized intersections in Hanoi is approximately 2,500 PCU/h/lane. However, considering a typical effective green ratio of 0.5, the actual lane capacity is constrained to roughly 1,250 veh/h/lane. Consequently, our chosen demands correspond to Volume-to-Capacity (V/C) ratios of approximately 0.4, 0.6, and 0.8. This allows us to evaluate the controller under free-flow, stable flow, and near-saturation conditions respectively, ensuring that the "High" demand scenario tests the system's limit without causing immediate, unrealistic gridlock.

The second reason is practical computing. SUMO is a microscopic simulator that models vehicles individually [2, 19]. As demand approaches the theoretical capacity limit, the computational cost increases non-linearly due to the complexity of resolving vehicle interactions in a gridlock state. By capping the demand at 1000 veh/hr/lane ($V/C \approx 0.8$), we ensure the simulation remains computationally feasible for running multiple seeds and training episodes on standard hardware, while still providing a rigorous stress test for the Reinforcement Learning agent.

4.3 Discussion: what the results mean

The results show a clear trade-off. Classical methods like Fixed-time and Webster are very efficient when traffic is light or regular. They follow a strict pattern that works well for free-flow traffic. RL, on the other hand, requires learning. At low demand, the "cost" of learning is that it performs slightly worse than the optimal fixed pattern.

However, the strength of RL appears at high demand (1000 veh/hr/lane). In heavy traffic, the network is chaotic. Fixed patterns cannot adapt to the changing jams. RL adapts its policy to minimize waiting time and maximize completion rate. This suggests that RL is most useful for busy city centers where traffic is heavy and unpredictable, while Fixed-time is better for suburbs with light traffic.

Teleportation is tracked as a diagnostic KPI. In SUMO, teleports happen when vehicles are stuck for too long. Tracking teleports helps ensure that the improvements at high demand are real and not just because cars are being removed from the simulation.

4.4 Limitations and Future Work

While the framework is robust, this study has limitations. First, the experiments use a synthetic grid network, not a real-world map. Second, we compared RL against basic methods. Future work should compare RL with more advanced adaptive controllers. We also aim to improve the RL training so it can perform better even at low demand.

References

- [1] Texas A&M Transportation Institute, “2019 Urban Mobility Report,” College Station, TX, USA, 2019.
- [2] D. Krajzewicz, J. Erdmann, M. Behrisch, and L. Bieker, “Recent Development and Applications of SUMO – Simulation of Urban Mobility,” *International Journal On Advances in Systems and Measurements*, vol. 5, no. 3&4, pp. 128–138, 2012.
- [3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [4] H. Wei, G. Zheng, V. Gayah, and Z. Li, “A Survey on Traffic Signal Control Methods,” *arXiv preprint arXiv:1904.08117*, 2019.
- [5] H. Wei, G. Zheng, H. Yao, and Z. Li, “IntelliLight: A Reinforcement Learning Approach for Intelligent Traffic Light Control,” in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, London, UK, 2018, pp. 2496–2505.
- [6] K. Abdulhai, R. Pringle, and G. Karakoulas, “Reinforcement Learning for True Adaptive Traffic Signal Control,” *Journal of Transportation Engineering*, vol. 129, no. 3, pp. 278–285, 2003.
- [7] I. Arel, C. Liu, T. Urbanik, and A. Kohls, “Reinforcement Learning-Based Multi-Agent System for Network Traffic Signal Control,” *IET Intelligent Transport Systems*, vol. 4, no. 2, pp. 128–135, 2010.
- [8] R. A. Wiering, “Multi-agent Reinforcement Learning for Traffic Light Control,” in *Proceedings of the 17th International Conference on Machine Learning (ICML)*, Stanford, CA, USA, 2000, pp. 1151–1158.
- [9] W. Genders and S. Razavi, “Using a Deep Reinforcement Learning Agent for Traffic Signal Control,” *Transportation Research Part C: Emerging Technologies*, vol. 68, pp. 1–20, 2016.
- [10] V. Mnih, K. Kavukcuoglu, D. Silver, et al., “Human-level Control Through Deep Reinforcement Learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

- [11] H. van Hasselt, A. Guez, and D. Silver, “Deep Reinforcement Learning with Double Q-learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, Phoenix, AZ, USA, 2016, pp. 2094–2100.
- [12] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, “Dueling Network Architectures for Deep Reinforcement Learning,” in *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, New York, NY, USA, 2016, pp. 1995–2003.
- [13] X. Liang, X. Du, G. Wang, and Z. Han, “Deep Reinforcement Learning for Traffic Light Cycle Control,” *IEEE Transactions on Vehicular Technology*, vol. 68, no. 2, pp. 1243–1253, 2019.
- [14] T. Chu, J. Wang, L. Codecà, and Z. Li, “Multi-Agent Deep Reinforcement Learning for Large-Scale Traffic Signal Control,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 3, pp. 1086–1095, 2020.
- [15] S. El-Tantawy, B. Abdulhai, and H. Abdelgawad, “Multiagent Reinforcement Learning for Integrated Network of Adaptive Traffic Signal Controllers,” *Transportation Research Part C: Emerging Technologies*, vol. 28, pp. 130–151, 2013.
- [16] E. van der Pol and F. A. Oliehoek, “Coordinated Deep Reinforcement Learners for Traffic Light Control,” in *Proceedings of the NIPS Workshop on Learning, Inference and Control of Multi-Agent Systems*, Barcelona, Spain, 2016.
- [17] Eclipse SUMO Documentation, “Simulation / Teleports (time-to-teleport),” 2024. [Online]. Available: https://sumo.dlr.de/docs/Simulation/Why_Vehicles_are_teleporting.html.
- [18] P. Varaiya, “The Max-Pressure Controller for Arbitrary Networks of Signalized Intersections,” in *Advances in Dynamic Network Modeling in Complex Transportation Systems*, S. V. Ukkusuri and K. Ozbay, Eds. New York, NY, USA: Springer, 2013, pp. 27–49.
- [19] M. Behrisch, L. Bieker, J. Erdmann, and D. Krajzewicz, “SUMO – Simulation of Urban Mobility: An Overview,” in *Proceedings of the Third International Conference on Advances in System Simulation (SIMUL 2011)*, Barcelona, Spain, 2011.
- [20] V. François-Lavet, P. Henderson, R. Islam, M. G. Bellemare, and J. Pineau, “An Introduction to Deep Reinforcement Learning,” *Foundations and Trends in Machine Learning*, vol. 11, no. 3–4, pp. 219–354, 2018.

- [21] P. Mirchandani and L. Head, “A Real-time Traffic Signal Control System: Architecture, Algorithms, and Analysis,” *Transportation Research Part C: Emerging Technologies*, vol. 9, no. 6, pp. 415–432, 2001.
- [22] P. Hunt, D. Robertson, R. Bretherton, and M. Royle, “The SCOOT On-line Traffic Signal Optimisation Technique,” *Traffic Engineering & Control*, vol. 23, no. 4, pp. 190–192, 1982.
- [23] D. Gartner, “OPAC: A Demand-Responsive Strategy for Traffic Signal Control,” *Transportation Research Record*, no. 906, pp. 75–81, 1983.
- [24] A. Stevanovic, *Adaptive Traffic Control Systems: Domestic and Foreign State of Practice*, NCHRP Synthesis 403, Transportation Research Board, Washington, DC, USA, 2010.
- [25] Y. Zhang, M. H. Abdoos, and A. Afshar, “Multi-Intersection Traffic Signal Control with Deep Reinforcement Learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, New York, NY, USA, 2020.
- [26] Transportation Research Board, *Highway Capacity Manual: A Guide for Multimodal Mobility Analysis*, 6th ed., Washington, D.C., USA, 2016.
- [27] H. C. Nguyen and K. Sano, “Determination of Saturation Flow at Signalized Intersections in Mixed Traffic Conditions in Hanoi, Vietnam,” *Journal of the Eastern Asia Society for Transportation Studies*, vol. 10, pp. 1558–1571, 2013.

A

**Implementation
details
described**

This appendix explains the logic of the code in a “code-to-language” way. The goal is to describe what each main module does with enough detail that a reader can rebuild the same system without seeing the original repository. The language is kept simple, but the meaning is technical and precise.

A.1 Environment core logic (SUMO + TraCI wrapper)

A.1.1 What the environment is responsible for

The environment is a bridge between two worlds: (1) SUMO, which simulates vehicle-level traffic flow, and (2) an RL agent, which expects a clean interface with `reset()` and `step()`. In every episode, the environment must: (i) start and stop SUMO safely, (ii) expose an observation vector in a fixed format (14D per TLS), (iii) apply timing actions with realistic safety constraints (min green, yellow, all-red), (iv) compute a global reward and KPIs, and (v) return diagnostics in `info` so failures (deadlock / teleport) can be analyzed.

A.1.2 Startup: configuration parsing and validation

At the start of an experiment, the environment reads configuration values such as: the network file, route files or route pools, traffic light IDs (TLS list), the center TLS ID, lane groups per TLS, downstream links for J0, and safety timing values. It then builds a discrete action table. If the action table is not given directly, it is generated by taking each cycle option in $\{60, 90, 120\}$ seconds and combining it with each split ratio in the set $\{0.30, 0.40, 0.50, 0.60, 0.70\}$ for NS. Each action is stored with fields `cycle_sec`, `rho_ns`, and `rho_ew`.

A strict validation step checks that `rho_ns + rho_ew` is approximately 1.0, because otherwise the green split would not match the cycle and would break timing logic. The environment also validates that minimum-green constraints are feasible for all actions: for every action, both NS and EW greens must be at least $g_{\min}$ after clipping.

A.1.3 Reset: selecting a route and launching SUMO

When `reset()` is called, the environment chooses a route file. If a route pool is used, it selects a route from the pool based on demand and seed (so experiments are reproducible). Then it launches SUMO using a configured binary and TraCI port. It sets the random seed, and it advances the simulation if a warm-up time is configured. During warm-up,

KPIs are not counted. Warm-up is used to avoid measuring unrealistic empty-network transient behavior.

A.1.4 Step: decoding actions into safe phase timing

When `step(actions)` is called, the environment receives either: (a) one action ID (single TLS mode), or (b) a mapping from `tls_id` to action ID (multi-TLS mode).

For each TLS, the environment maps the action ID to a timing plan $(C, \rho_{NS}, \rho_{EW})$. It computes green times using minimum-green clipping:

$$g_{NS}^{raw} = \text{round}(\rho_{NS} \cdot C), \quad g_{NS} = \max(g_{\min}, \min(g_{NS}^{raw}, C - g_{\min})), \quad g_{EW} = C - g_{NS}.$$

It then applies a deterministic safe phase sequence: NS green $\rightarrow$ NS yellow $\rightarrow$ all-red $\rightarrow$ EW green $\rightarrow$ EW yellow $\rightarrow$ all-red. This ensures safety and consistent clearance between conflicting movements.

The effective decision duration is computed as:

$$\Delta t = C + 2 \cdot y + 2 \cdot r,$$

where y is yellow duration and r is all-red duration. The environment advances SUMO for Δt seconds using step length 1 second. After each SUMO second, it collects lane measurements needed for queue and waiting time. At the end of the decision step, it builds the next 14D state for each TLS.

A.1.5 Measurements: queue, waiting, downstream occupancy, and global broadcast

Queue values are computed using a fixed rule such as counting stopped vehicles under a speed threshold (halt-speed threshold). Waiting time values are accumulated over time by reading SUMO waiting metrics (per lane, aggregated into N/E/S/W per TLS).

Downstream occupancy values are read only for the center TLS J0, using a configured list of downstream links (N/E/S/W). Satellite intersections set these four values to zero, but still keep the same state dimension to support parameter sharing.

Global features are computed from network-level counters, such as number of vehicles currently present and a global congestion scalar related to occupancy/spillback. Finally, normalization is applied if enabled.

A.1.6 Reward and termination

The environment computes a shared reward once per step. It computes total waiting time in the network during that step, and divides it by $N \cdot t_{ref}$ where N is number of vehicles and $t_{ref} = 60$. It then adds a spillback penalty term, which is the mean of squared downstream occupancy at J0 multiplied by α and scaled by $\Delta t / t_{ref}$.

If the configuration disables teleport penalty, teleports are tracked but not included in reward. The function returns the reward, the next state, a done flag, and an `info` dictionary. The `info` dictionary includes KPIs and diagnostic flags such as deadlock detection if enabled.

A.2 Normalization logic

A.2.1 What statistics are stored

The normalizer loads statistics from a JSON file. For dimensions 0–11 it stores mean and standard deviation. For dimensions 12–13 it stores min and max. This matches the state structure: (0–11) are local per-approach signals that vary widely and benefit from z-score, while (12–13) are broadcast global scalars that are naturally bounded and fit min–max scaling.

A.2.2 Transform rule

In a transform call, the normalizer: (1) standardizes dimensions 0–11 by z-score, (2) scales dimensions 12–13 by min–max, (3) applies clipping to cap outliers. This produces a stable numeric range for the neural network and helps avoid gradient explosions when rare congestion events create extremely large raw values.

A.3 KPI tracker logic

A.3.1 Counters and raw aggregation

The KPI tracker maintains counters for departed vehicles and arrived vehicles. It also tracks teleported vehicles. It accumulates total waiting time and total travel time over the measurement window (excluding warm-up). It also tracks per-step throughput and queue statistics, so reporting can show both averages and extreme behaviors.

A.3.2 Completion-corrected metrics

At the end, it computes:

$$\text{completion_rate} = \frac{\text{arrived}}{\text{departed}}, \quad \text{teleport_rate} = \frac{\text{teleported}}{\text{departed}}.$$

To reduce bias from unfinished trips, it computes corrected metrics by dividing time measures by $\text{arrived} \cdot \text{completion_rate}$, and it computes corrected throughput by dividing arrived by completion rate. These corrected metrics are then stored for reporting.

A.4 Agent logic (Dueling Double DQN with time-aware gamma)

A.4.1 Network structure

The agent creates an online network and a target network. Each network has two fully connected hidden layers and then splits into two heads. The value head outputs one scalar value $V(s)$. The advantage head outputs one advantage value per action $A(s, a)$. Q-values are computed by the standard dueling combination:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right).$$

A.4.2 Action selection

The agent uses epsilon-greedy exploration. With probability ϵ it samples a random action, and otherwise it chooses the action with maximum Q-value. If an allowed-action mask is provided, it sets disallowed actions to a very negative value before `argmax`, so they are never selected.

A.4.3 Replay buffer and time-aware discount storage

The replay buffer stores transitions. Each transition includes: state, action, reward, next state, done, γ , and optional diagnostic fields. Gamma is stored per transition because gamma depends on Δt (semi-MDP).

When time-aware gamma is enabled, gamma is computed as:

$$\gamma_t = \gamma_0^{\Delta t / t_{ref}},$$

and the computed γ_t is stored with the transition and used in the target update.

A.4.4 Double DQN update rule

When `update()` is called, the agent samples a batch. It computes the next action using the online network:

$$a^* = \arg \max_{a'} Q_{\theta}(s', a').$$

It then evaluates that action using the target network:

$$Q_{\theta-}(s', a^*).$$

The target is:

$$y = r + \gamma Q_{\theta-}(s', a^*) (1 - \text{done}).$$

The current estimate is $Q_{\theta}(s, a)$ for the chosen action. The loss is Huber loss between current and target. It backpropagates, clips gradients to a maximum norm of 10, and updates parameters with Adam. Every fixed number of steps, it copies online parameters to the target network.

A.5 Training loop logic

A.5.1 Main loop

Training runs for many episodes. Each episode: (1) resets the environment, (2) repeatedly reads state, chooses action, applies action, receives next state and reward, (3) stores transition in replay buffer (including per-step gamma), (4) periodically updates the network.

A.5.2 Exploration schedule and curriculum

Epsilon decays from a higher value to a lower value, so the agent explores more in early training and exploits more later. A curriculum chooses demand scenarios in phases. Early phases contain lower demand, and later phases contain higher demand. This stabilizes learning and reduces early collapse due to frequent deadlocks in heavy congestion.

A.6 Evaluation loop logic

A.6.1 Greedy policy evaluation

Evaluation loads the trained model and runs episodes with epsilon set to zero (greedy actions). It repeats the experiment for each demand and each seed, and records KPIs.

A.6.2 Baseline comparison

It also runs baseline controllers under the same timing constraints (same action set and safety rules). It aggregates mean and standard deviation across seeds for each KPI to ensure robust comparison.

B

**contribution
and log**

B.1 Division of work

This project was completed as a collaborative group effort where we split tasks and subsequently reviewed each other’s work to ensure quality and consistency. The workload was distributed across five primary areas of responsibility. First, we focused on the SUMO network setup and the generation of realistic route files. Second, we developed the TraCI environment wrapper alongside a robust KPI logging system to capture simulation data. Third, we implemented the Reinforcement Learning training code and conducted the necessary hyperparameter tuning runs. Fourth, we established the baseline implementations to ensure a fair and rigorous comparison with the RL agent. Finally, the team collaborated on the comprehensive report writing and technical formatting.

B.2 Short experiment diary

We maintained short technical notes during our experimental runs to track specific issues and their corresponding fixes. For instance, we recorded that one early run produced unstable rewards at high demand, which we later traced back to an incorrect downstream link mapping at the J0 intersection. We also observed that in a few specific random seeds, the network became close to gridlock under fixed-time control. Because teleport counts increased significantly in those instances, we decided to retain teleportation metrics as a critical diagnostic KPI. Furthermore, we noted that after strictly enforcing minimum green and clearance times, the simulation became much more stable and the results were significantly easier to compare across different policies. While this log is not a complete dataset of every event, it effectively demonstrates how we verified the system step by step during the implementation process.